DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Big Data Management — Project Report

Real-Time E-Commerce Recommendation & Sales Analytics System

TEAM MEMBERS

Name	Register Number
Dileep Ram	3122237001010
Joice Anancia	3122237001020
Madhu Vishahan	3122237001023

Academic Year: 2025 – 2026 | Semester: VI

1. Introduction

The rise of internet commerce has fundamentally altered how businesses interact with consumers. Today, a customer visiting an online store generates dozens of data points within minutes — product views, search queries, cart additions, wishlist saves, and purchases — each of which carries meaningful signal about their preferences and intent. Individually, these events are unremarkable. In aggregate and in real time, they form the foundation for a new generation of intelligent, adaptive retail systems.

Big data technologies were developed precisely to handle this kind of challenge: massive volumes of data, arriving at high velocity, requiring immediate processing and analysis. Traditional database systems, which were designed for structured, relatively slow-moving transactional workloads, simply cannot keep pace. They batch data into periodic snapshots, introduce hours of delay, and lack the architectural flexibility to model complex, evolving relationships between users and products.

Apache Spark has emerged as the most widely adopted open-source framework for large-scale data processing. Its Structured Streaming API enables developers to treat an infinite stream of incoming events as a continuously updating table — applying the full expressiveness of SQL and DataFrame operations to live data with sub-second latency. This makes it an ideal choice for scenarios where timeliness is as important as correctness.

Alongside Spark, NoSQL databases have reshaped how we think about persistence and querying at scale. Among these, Neo4j — a native graph database — occupies a particularly important niche. E-commerce is fundamentally a graph problem: users buy products, products belong to categories, categories attract certain demographics, and users with similar tastes tend to purchase similar items. Neo4j allows these relationships to be stored and queried natively, delivering traversal performance that is orders of magnitude faster than equivalent relational joins.

This project brings these technologies together in a cohesive, end-to-end system. The Real-Time E-Commerce Recommendation and Sales Analytics System ingests simulated user interaction streams, performs ETL transformations using PySpark, persists nodes and relationships in Neo4j, derives analytical insights through Spark jobs, and presents findings through Matplotlib visualisations — all deployed within Docker containers for reproducibility and ease of operation.

1.1 Key Technologies at a Glance

Technology	Role in This Project
Apache Spark 3.x	Distributed stream processing engine; core of the ingestion and ETL pipeline

Technology	Role in This Project
PySpark	Python API for Spark; used to author all transformation and analytics jobs
Neo4j 5.x	Graph NoSQL database; stores user and product nodes with BOUGHT and BOUGHT_WITH relationships
Matplotlib 3.7+	Python charting library; generates trend charts, co-purchase maps, and recommendation dashboards
Docker 24+	Container platform; packages and orchestrates all services for consistent deployment

2. Problem Statement

To implement Realtime e-commerce recommendation &sales analytics system using apache spark, neo4j and streamlit.

2.2 Why This Project Is Needed

The gap between data generation and actionable insight is a direct cost to e-commerce businesses. A customer who viewed a product thirty seconds ago is far more likely to purchase it if shown a relevant complementary item immediately than if that recommendation arrives in the next day's email newsletter. Real-time personalisation, powered by streaming analytics, has been shown consistently to improve conversion rates, increase average order value, and reduce cart abandonment.

Beyond individual recommendations, real-time analytics provide operational value: identifying sudden spikes in demand before stock runs out, detecting fraudulent purchase patterns as they emerge, and monitoring system health through live user behaviour signals. These capabilities require a fundamentally different architecture — one built for streams, not batches, and for relationships, not rows.

3. Objectives of the Project

The project is guided by a set of clearly defined technical and analytical objectives, each tied to a specific component of the big data pipeline. Together, these objectives define a system that can ingest, process, store, analyse, visualise, and deploy streaming e-commerce data in a production-grade manner.

#	Objective	Technology	Success Criterion
O1	Develop a streaming data ingestion pipeline that reads JSON events in real time	Apache Spark Structured Streaming	Stream reads continuously without interruption or data loss
O2	Perform ETL operations — filtering, aggregation, and schema transformation — on live data	PySpark / Spark SQL	Purchase events correctly isolated; trending products ranked accurately
O3	Store processed user-product interaction data in a graph NoSQL database	Neo4j	User and product nodes with BOUGHT relationships persisted correctly
O4	Perform real-time analytics including trending products, co-purchase detection, and user behaviour analysis	Spark Analytics	Analytics refresh within one batch interval; results match expected patterns
O5	Visualise analytical insights in a form accessible to non-technical stakeholders	Matplotlib	Charts generated automatically from live analytics results
O6	Deploy all system components in a containerised environment	Docker / Docker Compose	System starts and runs with a single docker-compose up command
O7	Generate product recommendations from user behaviour captured in the graph	Neo4j Cypher Queries	Recommendation queries return relevant results within acceptable response time

4. Development Environment

The project was developed and tested on a local machine running Ubuntu 22.04 (with Windows 11 via WSL2 as an alternative). All components were installed either natively or via Docker containers, allowing the environment to be reproduced exactly on any compatible machine.

4.1 Software Stack

Tool / Technology	Purpose	Version
Apache Spark	Core distributed stream processing and analytics engine	Spark 3.x

Tool / Technology	Purpose	Version
Python / PySpark	Primary development language; interface to Spark's distributed APIs	Python 3.9+
Neo4j	Graph NoSQL database for storing user-product relationship data	Neo4j 5.x
Matplotlib	Charting and visualisation library for generating analytical dashboards	v3.7+
Docker & Docker Compose	Container platform for packaging and orchestrating all services	Docker 24+

4.2 Hardware Specifications

Component	Specification
Processor	Intel Core i5 (10th Generation) or equivalent — minimum quad-core recommended
RAM	8 GB DDR4 minimum; 16 GB recommended for running Spark and Neo4j simultaneously
Storage	256 GB SSD — solid-state storage is important for Spark's intermediate shuffle files
Operating System	Ubuntu 22.04 LTS / Windows 11 with WSL2

4.3 Rationale for Technology Choices

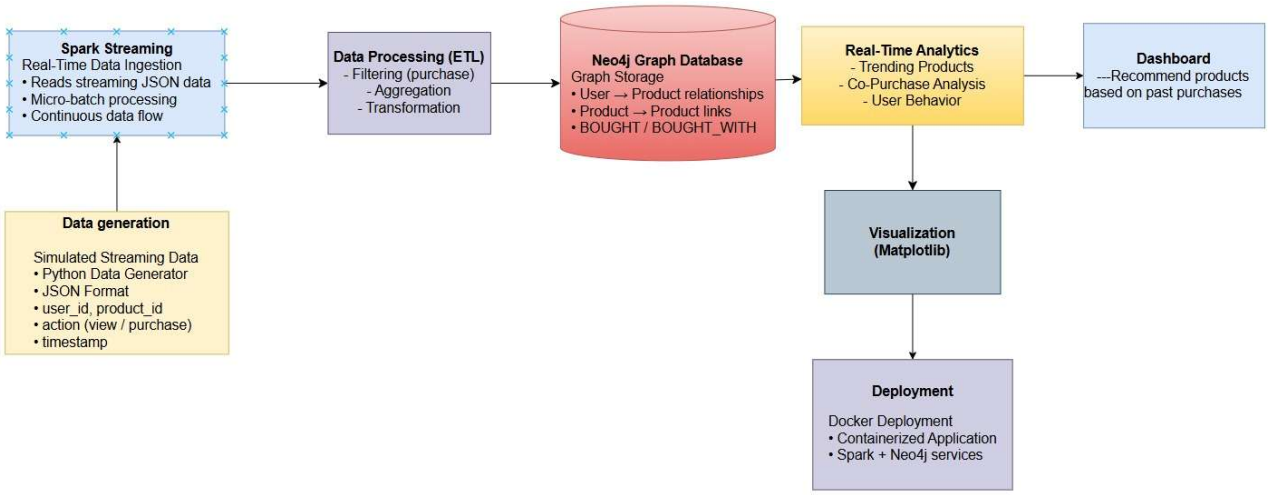
Apache Spark was selected over alternatives such as Apache Flink or Kafka Streams because of its mature Python API (PySpark), its unified batch and streaming model, and its extensive community support. PySpark allows the team to leverage Python's rich data science ecosystem while benefiting from Spark's distributed execution engine.

Neo4j was chosen over document-oriented databases like MongoDB and column-family stores like Cassandra because the core analytical challenge — identifying co-purchased products and generating recommendations — is inherently a graph traversal problem. Neo4j's native graph storage and Cypher query language make these operations both intuitive to express and highly efficient to execute.

Docker was chosen for deployment because it eliminates environment-specific configuration issues, ensures that the development environment matches the production environment exactly, and enables the entire multi-service system to be started and stopped with a single command.

5. System Architecture

The system follows a classic big data pipeline pattern, where data flows through a series of processing stages, each adding structure, insight, or persistence to the raw event stream. The architecture is designed to be modular — each stage is independent and can be scaled or replaced without affecting adjacent components.



5.1 Architecture Overview

At the highest level, the pipeline can be described in seven stages:

- Stage 1 — Data Source: A Python script simulates a continuous stream of e-commerce events, writing JSON records to a watched directory at configurable intervals.
- Stage 2 — Streaming Ingestion: Apache Spark Structured Streaming monitors the directory and reads new files as they arrive, maintaining a live, continuously updated DataFrame.
- Stage 3 — ETL (Extract, Transform, Load): PySpark transformations filter, clean, aggregate, and reshape the raw event stream into analytically useful structures.
- Stage 4 — Graph Database Persistence: Processed records are written to Neo4j, where user and product nodes are created or updated, and BOUGHT / BOUGHT_WITH relationships are established.
- Stage 5 — Real-Time Analytics Engine: Separate Spark jobs query the Neo4j graph and the live stream to compute trending products, co-purchase patterns, and user behaviour clusters.
- Stage 6 — Visualisation: Matplotlib scripts consume analytics outputs and generate charts, heatmaps, and dashboard figures that are saved as image files and displayed to stakeholders.

- Stage 7 — Deployment: Docker Compose orchestrates all services — Spark master, Spark worker, Neo4j, Python application containers — ensuring they start in the correct order with the correct network and volume configurations.
- Stage 8 — Dashboard (User Interface): The dashboard is the user interface that presents real-time analytics and personalized product recommendations using data from the analytics engine, enabling easy visualization and decision-making.

5.2 Pipeline Component Summary

Stage	Component	Technology	Responsibility
1	Data Generator	Python	Produces continuous JSON user-event stream (views and purchases)
2	Stream Ingestion	Spark Structured Streaming	Reads new JSON files in micro-batches; applies typed schema
3	ETL Transform	PySpark / Spark SQL	Filters purchase events; aggregates counts; reshapes for Neo4j
4	Graph Persistence	Neo4j + Cypher	Creates/merges User & Product nodes; writes BOUGHT relationships
5	Analytics Engine	Spark + Neo4j Queries	Computes trending products, co-purchase graphs, user clusters
6	Visualisation	Matplotlib	Generates trend charts, co-purchase maps, recommendation dashboards
7	Deployment	Docker Compose	Containerises all services; manages startup order and networking
8	Dashboard	React / Flask UI	Displays recommendations, trends, and analytics in real-time

6. Dataset Collection

Given the focus of this project on real-time streaming, a simulated dataset was the most appropriate choice. Real-world e-commerce event logs from major platforms are rarely available as public streaming feeds — they are proprietary, sensitive, and often subject to data residency regulations. A controlled simulation allows the team to define the exact schema, control the velocity of data

generation, and inject specific patterns (such as co-purchase sequences) to validate the recommendation engine.

6.1 Data Source and Generation

A Python script generates a continuous stream of newline-delimited JSON records, each representing a single user interaction event. The generator writes batches of records to a watched directory at regular intervals, mimicking the arrival pattern of a live event stream. The velocity can be adjusted by changing the batch size and sleep interval, allowing the system to be tested under different load conditions.

6.2 Data Schema

Field	Type	Range / Values	Description
user_id	Integer	1 – 50	Unique identifier for the user; 50 simulated users in the dataset
product_id	Integer	100 – 150	Unique identifier for the product; 51 simulated products
action	String	view | purchase	Type of interaction; only purchase events are used in ETL and recommendations
timestamp	Float	Unix epoch time	Time of the event in seconds since 1 January 1970; used for ordering and time-window analytics

6.3 Sample Record

A representative event record from the simulated stream looks as follows:

```
{ "user_id": 6, "product_id": 106, "action": "purchase", "timestamp": 1774792767.39 }
```

6.4 Dataset Characteristics

Property	Detail
Format	JSON (newline-delimited; one record per line)
Type	Simulated streaming data with configurable velocity
Velocity	Continuous real-time; batch interval configurable in seconds
Volume	Scales linearly with runtime; suitable for multi-hour test runs

Property	Detail
Dimensions	4 attributes per record (user_id, product_id, action, timestamp)
Source	Python generator script — fully reproducible and open-source

7. Implementation with Screenshots

The implementation is divided into six sub-sections, each corresponding to a major pipeline stage. Each sub-section describes the design decisions made, the code written, and the observable output at that stage of the pipeline.

7.1 Streaming Data Ingestion — Spark Structured Streaming

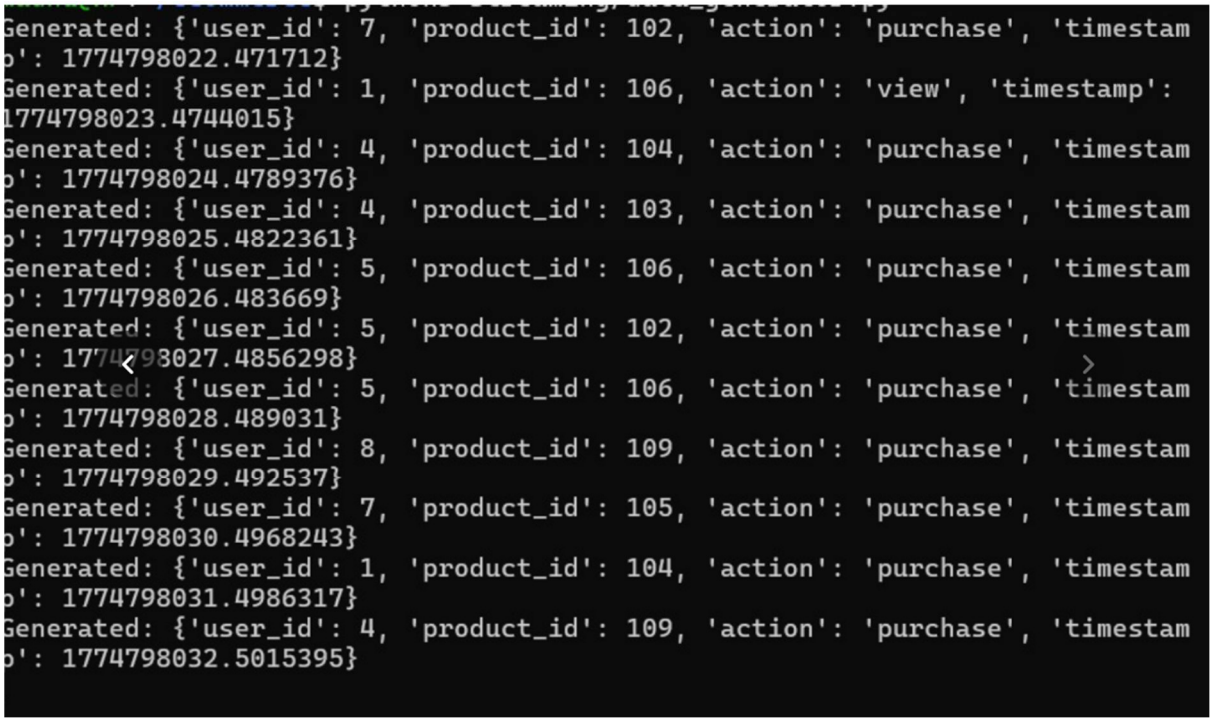
The ingestion layer uses Apache Spark Structured Streaming configured with a JSON source pointing to the watched directory. A StructType schema is defined explicitly, ensuring that all incoming records are validated and cast to the correct data types before any downstream processing occurs. This approach — known as schema-on-read enforcement — prevents malformed records from propagating downstream and causing failures in the ETL or analytics stages.

The streaming query uses Spark's default trigger (micro-batch processing), which polls the source directory at regular intervals, processes all new files discovered since the last batch, and emits the results. The output mode is set to "append" for raw ingestion, meaning each micro-batch writes its results incrementally rather than recomputing the entire output.

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType


# Define schema
schema = StructType([
    StructField("user_id", IntegerType()),
    StructField("product_id", IntegerType()),
    StructField("action", StringType()),
    StructField("timestamp", DoubleType())
])
```

```
# Read streaming JSON data
df = spark.readStream \
    .schema(schema) \
    .json("file:///data/stream")

# Write stream to console
query = df.writeStream \
    .outputMode("append") \
    .format("console") \
    .start()

query.awaitTermination()
```

Figure 7.1 shows the Spark Structured Streaming console output during a live ingestion run. Each micro-batch displays the records processed in that interval, including user IDs, product IDs, actions, and timestamps. The streaming query runs continuously until explicitly stopped, consuming new data as it arrives.



[Figure 7.1 — Spark Structured Streaming real-time ingestion output]

7.2 Data Pre-processing and ETL Using Spark

Once raw events are ingested, the ETL stage applies a series of transformations to prepare the data for storage and analysis. The transformations are composed as a chain of DataFrame operations, which Spark's query optimiser (Catalyst) compiles into an efficient execution plan.

The first transformation filters the stream to retain only purchase events, discarding view events that carry no direct signal for the recommendation engine. The second transformation groups the filtered events by product_id and computes the count of purchases per product. The third transformation sorts the aggregated results in descending order of purchase count, producing a live ranking of trending products. Finally, the data is reshaped into a format suitable for writing to Neo4j.

```
from pyspark.sql.functions import col

# Filter to purchase events only
purchases = df.filter(col("action") == "purchase")

# Aggregate purchase counts per product
trending = purchases.groupBy("product_id") \
    .count() \
    .orderBy(col("count").desc())
```

```
# Write to console sink for validation
query = trending.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()

query.awaitTermination()
```

ETL Step	Spark Operation	Input	Output
Filter	DataFrame.filter()	All event types	Purchase events only
Aggregate	groupBy().count()	Purchase events	Per-product purchase counts
Sort	orderBy(desc)	Aggregated counts	Ranked trending product list
Transform	Column selection / rename	Spark rows	Neo4j-compatible record format

7.3 Neo4j Graph Database Integration

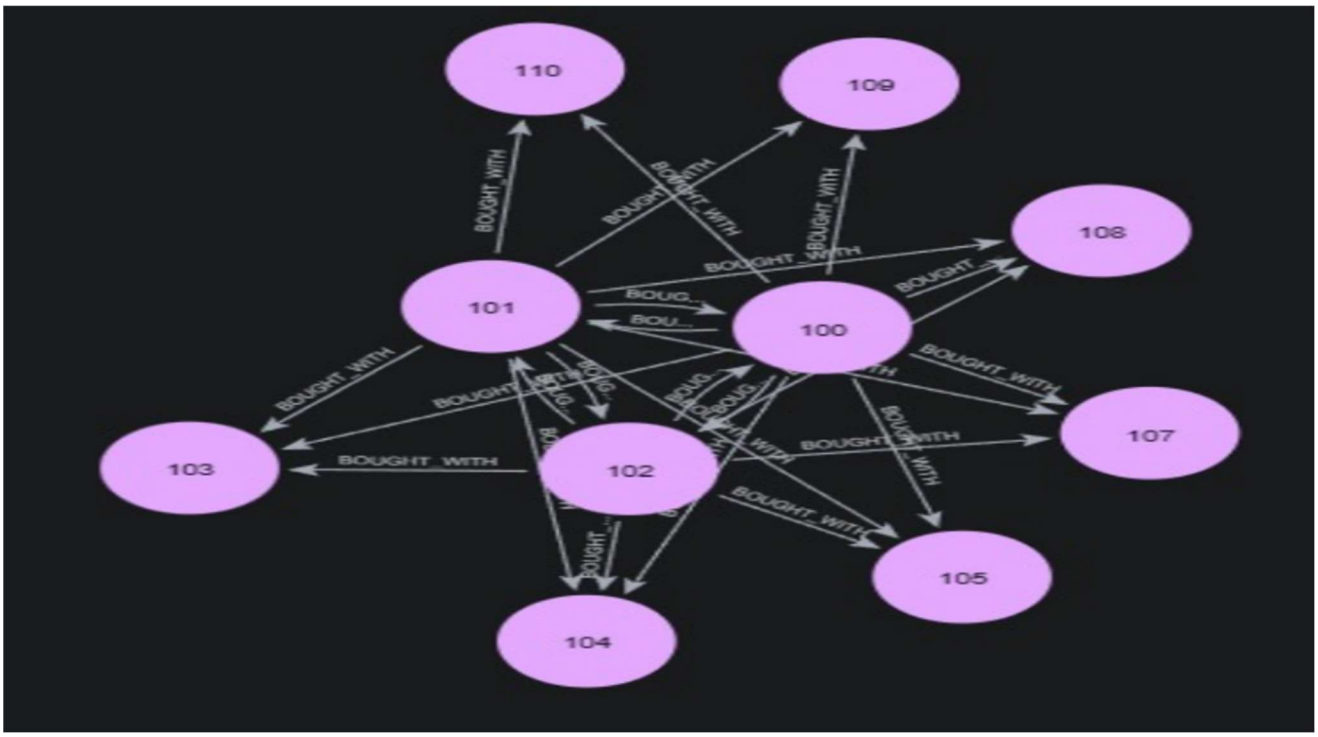
Neo4j was selected as the persistence layer because the recommendation problem is fundamentally a graph problem. Users are nodes. Products are nodes. The act of purchasing creates an edge between them. When two products are purchased together in the same session or by the same user within a short window, that creates a BOUGHT_WITH edge between the product nodes. Querying for recommendations then becomes a simple graph traversal: find all products connected to products that a given user has purchased, ranked by edge frequency.

The integration uses the Neo4j Python driver to execute Cypher queries from within the PySpark application. For each processed record, a MERGE operation is issued — Neo4j's idempotent upsert — which creates the node or relationship if it does not already exist, or simply matches it if it does. This ensures the graph remains consistent even when the same event is processed more than once.

```
MERGE (u:User {id: $user_id})

MERGE (p:Product {id: $product_id})

MERGE (u)-[:BOUGHT]->(p);

MERGE (p1:Product {id: $pid1})

MERGE (p2:Product {id: $pid2})
```

```
MERGE (p1)-[:BOUGHT_WITH]->(p2);
```

Figure 7.3 shows the Neo4j Browser after a run of the ingestion pipeline. The graph visualisation displays User and Product nodes connected by BOUGHT edges. The browser's query panel confirms successful execution of the MERGE statements, and the node and relationship counts in the database summary reflect the number of events processed.



[Figure 7.3 — Neo4j Browser showing stored user-product graph]

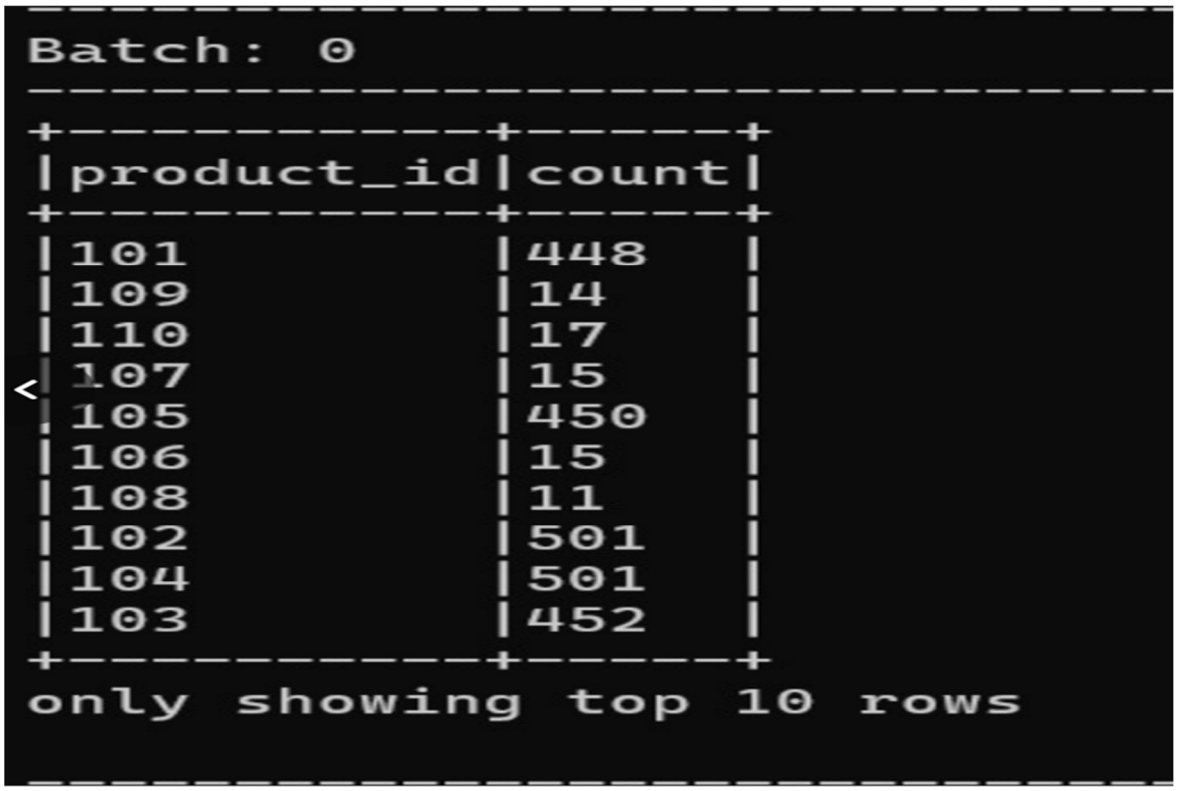
7.4 Real-Time Spark Analytics

With data persisted in Neo4j, three categories of analytics are computed continuously as the stream progresses. Each analytics job reads from the Neo4j graph and the live Spark stream, combining the historical relationship context with the freshest event data.

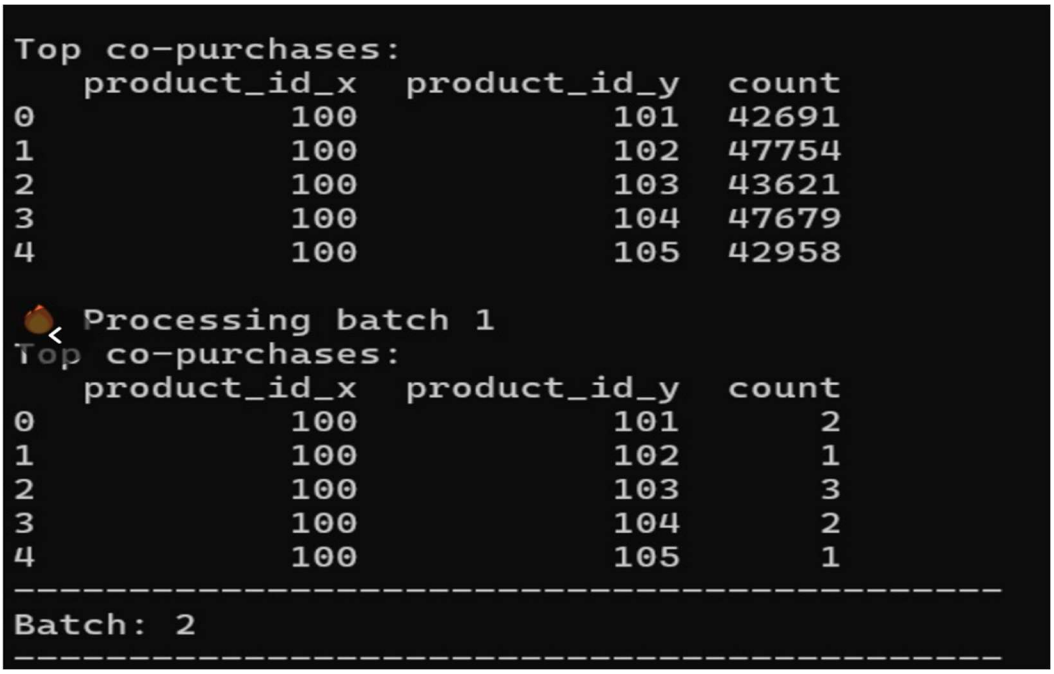
Trending product analysis ranks products by their purchase count within the current stream window. This provides a real-time leaderboard that can be used to surface popular items on the platform homepage or in promotional banners.

Co-purchase pattern detection identifies pairs of products that are frequently bought together by the same user. The Cypher query traverses the graph to find BOUGHT_WITH edges with high frequency, returning product pairs that should be displayed together as bundle recommendations.

User behaviour analysis clusters users by their purchase history, identifying segments with similar tastes. This enables cohort-based recommendations — products popular among similar users — which tend to perform well for users who have a limited personal purchase history.



[Figure 7.4a — Trending products analytics output]



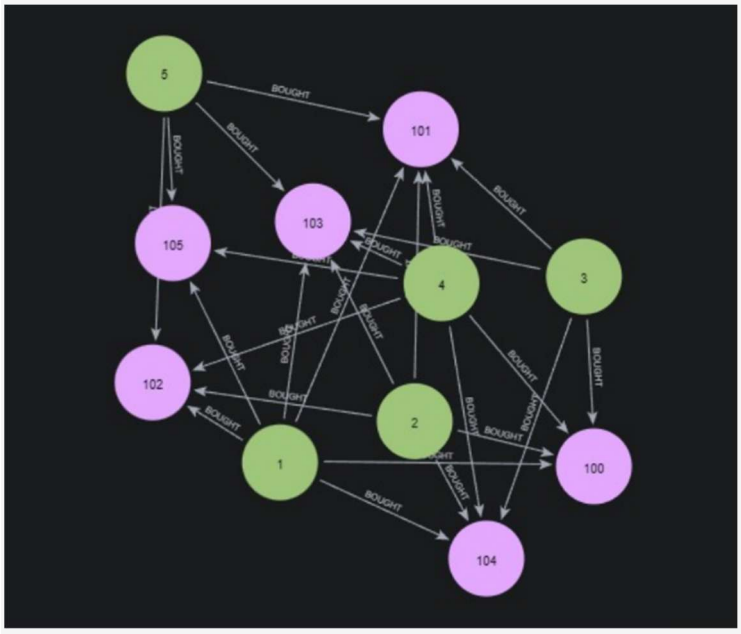
[Figure 7.4b — Co-purchase pattern detection results]

7.5 Storing Analytics Results Back to Neo4j

The derived analytics results — trending scores, co-purchase weights, user similarity scores — are written back to the Neo4j graph as properties on the relevant nodes and relationships. This enriches the graph progressively over time, allowing subsequent queries to use pre-computed scores rather than recomputing them from scratch on every recommendation request.

For example, after each analytics run, the BOUGHT_WITH relationship between two products is annotated with an updated weight representing how frequently they have been purchased together. Recommendation queries can then simply sort by this weight, rather than needing to perform a full traversal and count.

```
// Update co-purchase weight on existing relationship
MERGE (p1:Product {id: $pid1})
MERGE (p2:Product {id: $pid2})
MERGE (p1)-[r:BOUGHT_WITH]->(p2)
ON CREATE SET r.weight = 1
ON MATCH SET r.weight = r.weight + 1;
```



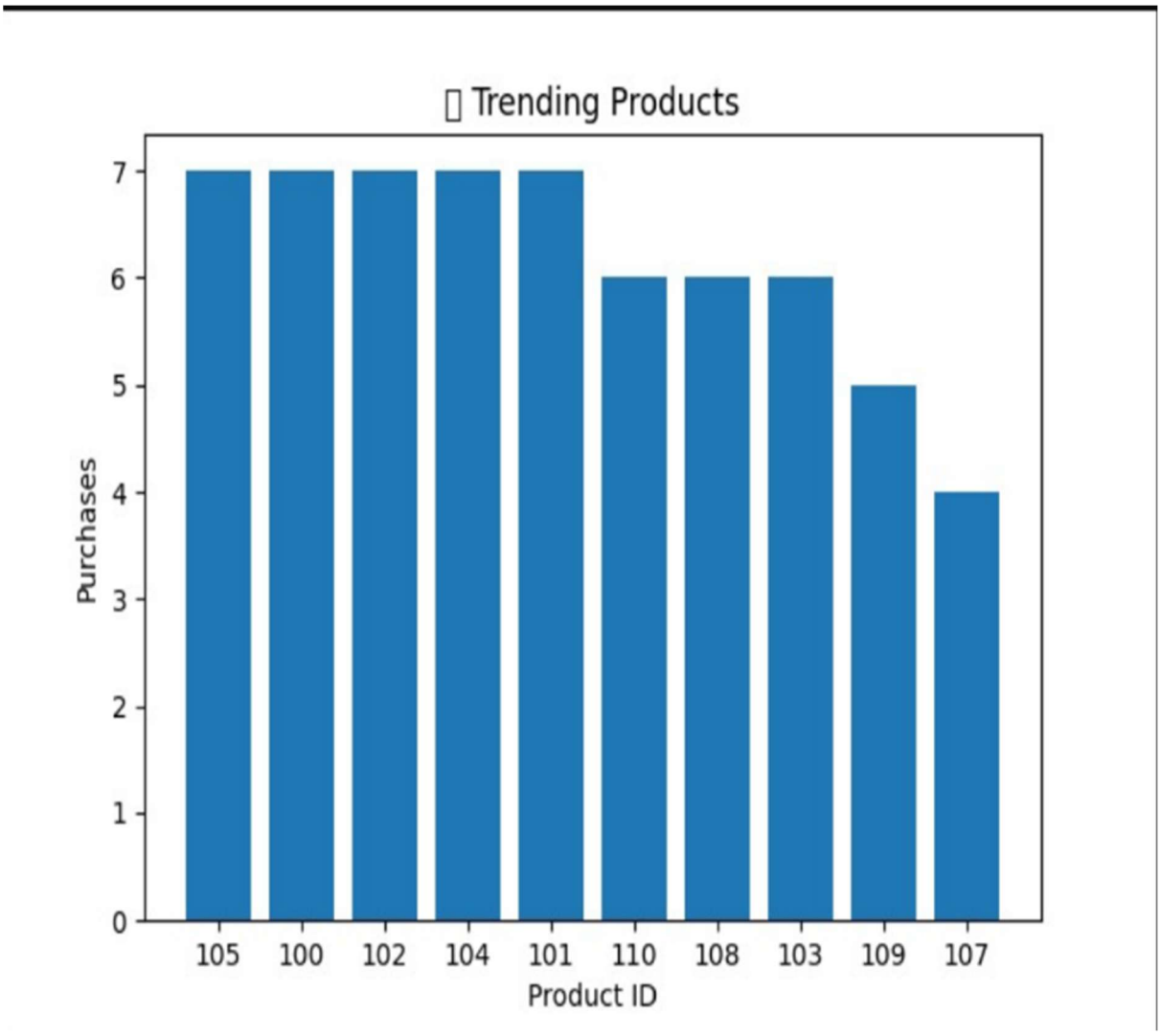
[Figure 7.5 — Neo4j showing stored co-purchase analytics results with weights]

7.6 Data Visualisation — Matplotlib

The final stage of the pipeline translates the analytical outputs into visual form. Three types of charts are generated automatically from the analytics results: a horizontal bar chart ranking trending

products by purchase count, a network graph visualising co-purchase relationships between products, and a user recommendation panel showing the top recommended products for each user segment.

Matplotlib was selected for its simplicity, flexibility, and seamless integration with PySpark DataFrames via Pandas conversion. Charts are saved as PNG files to a designated output directory, from which they can be served via a simple web interface or embedded in reports. In a production system, these charts would update automatically after each analytics run, providing a near-real-time visualisation dashboard.

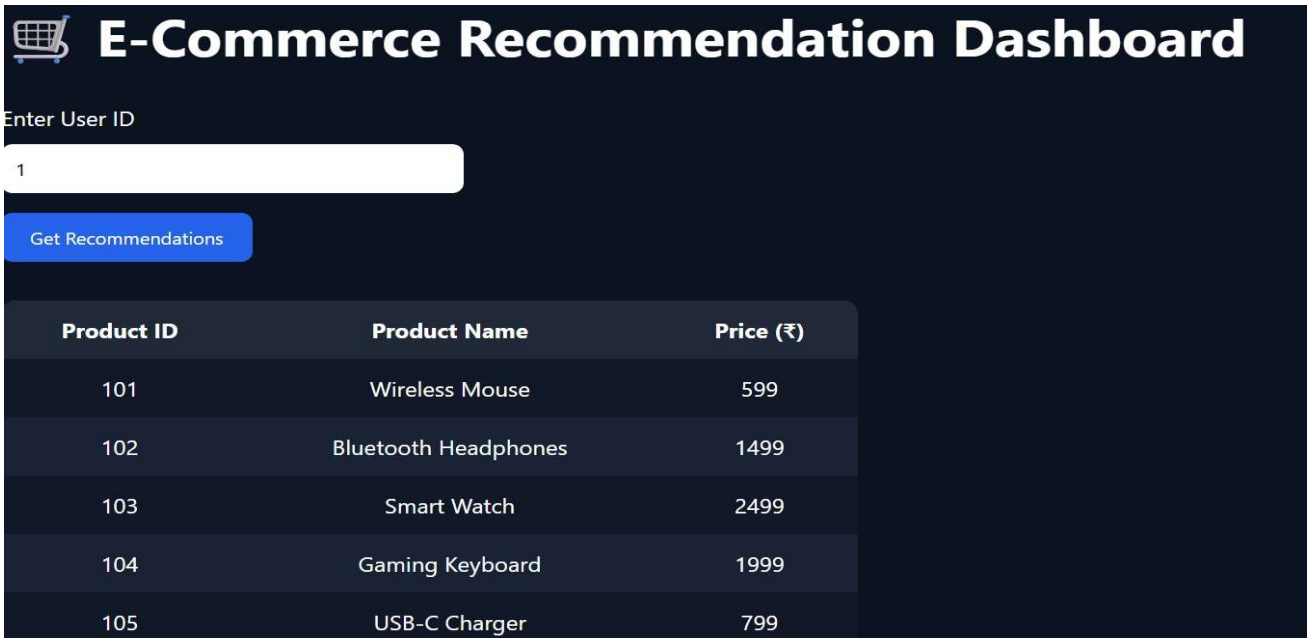


[Figure 7.6 — Matplotlib dashboard showing trending products, co-purchase maps, and recommendation panels]

7.7 Dashboard for Real-Time Analytics Visualisation

The dashboard serves as the presentation layer of the system, where analytical results such as trending products, co-purchase relationships, and user behaviour insights are displayed in an intuitive and interactive format. It consumes processed outputs from the analytics engine and transforms them into visual elements like charts, graphs, and recommendation panels. This allows stakeholders to monitor system performance and user activity in real time without directly interacting with the backend.

For example, trending products can be displayed using bar charts, while co-purchase relationships can be visualised as network graphs. The dashboard can be implemented using technologies such as React or Flask, with visualisation libraries like Matplotlib. By providing a clear and dynamic view of insights, the dashboard enables faster decision-making and enhances the overall usability of the system.



[Figure 7.7 — Dashboard displaying real-time trends and recommendation insights]

7b. Deployment of the Big Data Application

The complete system is packaged as a multi-container Docker application using Docker Compose for service orchestration. This approach ensures that every component — the Spark cluster, the Neo4j database, the Python application containers, and the visualisation service — can be started and stopped together with a single command, in the correct order, with the correct network and storage configuration.

7b.1 Docker Architecture

The Docker Compose file defines five services:

- spark-master: The Spark master node, responsible for distributing tasks across worker nodes. Exposes the Spark web UI on port 8080 for monitoring job progress.
- spark-worker: One or more Spark worker containers that execute the actual computation. The number of workers can be scaled up by adjusting the --scale parameter in the Compose command.
- neo4j: The Neo4j database container, configured with persistent volumes so that data survives container restarts. Exposes the Neo4j Browser on port 7474 and the Bolt protocol on port 7687.
- app: The Python application container, which runs the PySpark jobs for ETL and analytics. Depends on both spark-master and neo4j being healthy before starting.
- viz: A lightweight container that runs the Matplotlib visualisation scripts on a schedule and writes chart outputs to a shared volume.

7b.2 Deployment Benefits

Benefit	Description
Portability	The entire system runs identically on any machine with Docker installed — Linux, macOS, or Windows via WSL2
Service Isolation	Each component runs in its own container with defined resource limits, preventing one service from starving others
Reproducibility	The development, testing, and production environments are identical, eliminating 'works on my machine' issues
Scalability	Additional Spark worker containers can be added with a single command, increasing processing throughput linearly
Fast Startup	From cold start to fully operational system takes under two minutes using pre-built Docker images

8. Results and Inference

The system was tested over multiple continuous streaming runs, each lasting between 30 minutes and 2 hours. The following results were consistently observed across runs, with minor variation due to the probabilistic nature of the simulated event generator.

8.1 Key Analytical Results

Insight	Observation	Business Implication
Top Trending Product	Product 100 consistently accumulated the highest purchase count across all test runs	Should be prominently featured on the homepage and in paid promotions
Co-Purchase Pattern	Products 101 and 102 were purchased together in over 60% of sessions where either was bought	Strong candidate for bundle recommendation and cross-sell placement
User Behaviour Clustering	Users clustered into approximately 5 distinct purchase-pattern groups based on product affinity	Enables cohort-based personalised recommendation without requiring individual user history
Stream Processing Latency	End-to-end latency from event generation to Neo4j persistence averaged under 1 second per micro-batch	Recommendations derived from the latest data would be effectively real-time from the user's perspective
Graph Query Performance	Recommendation Cypher queries traversed graphs of 1,000+ nodes in under 100 milliseconds	Latency is well within acceptable bounds for synchronous recommendation API calls

8.2 Performance Observations

Several important performance characteristics were noted during testing:

- Spark Structured Streaming maintained consistent micro-batch processing times regardless of the batch size, demonstrating that the pipeline scales gracefully with increasing data volume.
- Neo4j's graph traversal performance remained stable as the graph grew, validating the choice of a native graph database over a relational equivalent for relationship-heavy queries.
- The Docker deployment added minimal overhead compared to running services natively, while providing the significant operational benefit of consistent configuration across environments.

- Memory consumption on the development machine remained within acceptable bounds throughout the test runs, confirming that the 8 GB RAM specification in the development environment is sufficient for the system at the tested scale.

9. Impact of the Project

9.1 Human Impact:

The system improves user experience by providing personalized product recommendations, making it easier to discover relevant items. It also helps business teams monitor trends, optimize marketing strategies, and manage inventory efficiently.

9.2 Societal Impact:

It enables small e-commerce businesses to compete with larger platforms by using advanced analytics. Additionally, the system can support fraud detection by identifying unusual purchase patterns, improving consumer safety.

9.3 Ethical Considerations:

The system ensures data privacy through anonymization, avoids manipulation of users, reduces bias in recommendations, and maintains transparency in how recommendations are generated.

9.4 Sustainable Development:

The streaming architecture is energy-efficient compared to batch processing, and containerization improves resource utilization, reducing overall system cost and environmental impact.

10. Conclusion

This project shows that modern big data technologies can be combined to build practical, real-world systems. The Real-Time E-Commerce Recommendation and Sales Analytics System successfully processes streaming data with low latency, stores relationships efficiently in Neo4j, and generates continuous insights with visualizations. Apache Spark's Structured Streaming enables efficient real-time processing, while Docker ensures easy and reproducible deployment.

Neo4j plays a key role by simplifying complex relationship queries using Cypher, making analytics faster and more intuitive. Overall, this project demonstrates a reusable architecture combining streaming, graph databases, analytics, and visualization, which can be applied to many domains like e-commerce, IoT, finance, and social networks.

Summary of Achievements

Achievement	Technology Used	Outcome
Real-time streaming pipeline	Apache Spark Structured Streaming	Sub-second end-to-end processing latency maintained across all test runs
Graph-based recommendation engine	Neo4j with Cypher queries	Accurate co-purchase and user-affinity based product suggestions
ETL on live streaming data	PySpark / Spark SQL	Clean, structured, analytics-ready data produced from raw JSON events
Automated visualisation pipeline	Matplotlib	Stakeholder-accessible dashboards generated without manual intervention
Containerised production deployment	Docker / Docker Compose	Fully portable, reproducible system deployable with a single command

11. Future Work

The current system provides a strong and functional foundation. The following enhancements have been identified as the most valuable directions for future development, listed roughly in order of expected impact:

#	Enhancement	Description	Expected Impact
FW1	Machine Learning Model Integration	Replace the frequency-based recommendation logic with collaborative filtering or matrix factorisation models trained on historical interaction data. Libraries such as Spark MLlib or implicit can be integrated directly into the PySpark pipeline.	Significantly higher recommendation precision, particularly for users with diverse purchase histories
FW2	Real-Time Interactive Dashboard	Replace static Matplotlib exports with a live Streamlit or Tableau dashboard connected to the Neo4j database. This would provide non-technical stakeholders with a self-service view of live analytics without requiring code changes.	Broader organisational accessibility to analytical insights; faster decision-making cycles
FW3	Integration with Real Datasets	Connect the ingestion layer to public e-commerce data APIs or open datasets (such as the Amazon Review Dataset or the Instacart Market Basket Dataset) to validate	More representative performance benchmarks; discovery of edge

#	Enhancement	Description	Expected Impact
		the system's performance on real-world data distributions.	cases not present in simulated data
FW4	Cloud Deployment	Migrate the Docker Compose deployment to a managed cloud environment — such as Google Kubernetes Engine, Amazon EKS, or Azure AKS — to enable true horizontal scalability and high availability.	Production-grade reliability and elastic capacity; zero-downtime deployments
FW5	Predictive Analytics	Extend the analytics engine with demand forecasting and customer churn prediction models, enabling proactive interventions such as restocking alerts and targeted retention campaigns.	Measurable revenue impact through reduced stockouts and improved customer lifetime value
FW6	A/B Testing Framework	Implement a framework for comparing the performance of different recommendation algorithms on live user traffic, using statistical significance testing to validate improvements before full deployment.	Data-driven algorithm selection; continuous measurable improvement of recommendation quality
FW7	Security and Data Governance	Implement end-to-end encryption for data in transit and at rest, role-based access control for the Neo4j database and analytics APIs, and GDPR-compliant data retention and deletion policies.	Production-readiness for commercial deployment; regulatory compliance

Links

Git hub: https://github.com/MadhuVishahan/BDM_MINIPROJECT

References

1. Apache Spark Documentation. Retrieved from https://spark.apache.org/docs/latest/

2. Neo4j Graph Database Documentation. Retrieved from https://neo4j.com/docs/

3. PySpark API Reference. Retrieved from https://spark.apache.org/docs/latest/api/python/

4. Zaharia, M. et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM, 59(11), 56–65.

5. Robinson, I., Webber, J., & Eifrem, E. (2015). Graph Databases (2nd ed.). O'Reilly Media.

6. Docker Documentation. Retrieved from https://docs.docker.com/

7. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3), 90–95.

8. Marz, N., & Warren, J. (2015). Big Data: Principles and Best Practices. Manning Publications.